Presentation Script

What to show, what to type and what to say, step by step

Mihir Bhati (24BCE5341) · Ahana Sharan (24BCE5334)

About 20 minutes, plus questions

How to use this script. Each step has: **show** (what is on screen), **type** (the command, if any), **say** (lines you can speak; use your own words once you know them) and **if asked** (a likely question). Steps marked *optional* can be skipped if time is short. Rehearse the whole thing once before the real demo.

Before the demo (10 minutes earlier)

☐ Laptop plugged in; Wi-Fi not needed (everything runs offline).
☐ Open the **Ollama** app from the Start menu (it runs the AI model).
☐ Open **PowerShell** and type `cd D:\MyClaudeProject\nlp`.
☐ Type `ollama list` and check `qwen2.5:3b` is listed.
☐ Run `python scripts/demo.py --project P1` once, so the model is warmed up (the first answer is slow).
☐ Open in File Explorer: `reports\figures` (the charts) and `reports\guide` (the documents).
☐ Keep **How Our System Works.pdf** open in a second window as your backup notes.

If any command fails during the demo, it is almost always because Ollama is closed. Open it, wait 10 seconds and run the command again. Say: "The AI model runs locally in Ollama; let me start it."

The flow at a glance

#	Step	Time	What it proves
1	The problem and the idea	2 min	you understand what you built
2	How the system works	3 min	the agent, the 12 designs, qwen's role
3	Live demo of the finished pipeline	2 min	it works
4	One test, step by step (walkthrough)	3 min	how every test is done
5	The agent choosing a design	2 min	the agentic part
6	The results and the comparisons	4 min	it is better, and where it is not
7	Proof the numbers are real	2 min	results can be checked independently
8	Built it yourself (optional)	2 min	you understand it well enough to rebuild it
9	Honest limits and next step	1 min	maturity and honesty
10	Close, then questions	1 min	

Step 1. The problem and the idea (2 min)

Show: the first page of How Our System Works.pdf, or just talk.

Say:

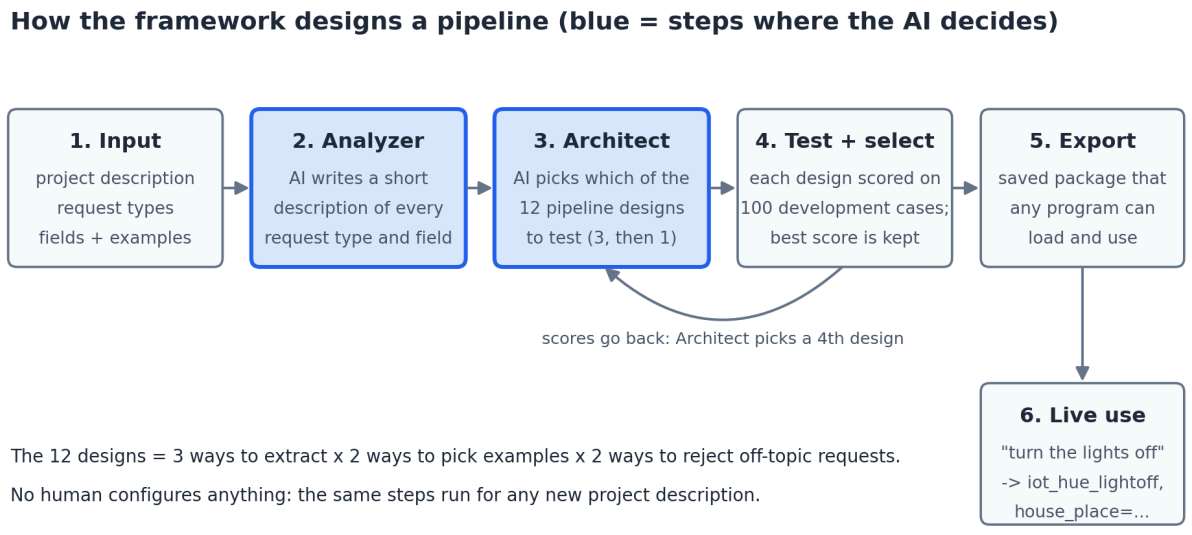
> "Every app that understands users, like a home assistant or a hospital bot, needs an NLP pipeline. It turns a sentence like 'turn off the bedroom lights' into something a program can act on: the intent, lights off, and the detail, bedroom."
> "There are two ways to build that. You can train a model on examples, like XLM-R, the model the MASSIVE dataset paper used. Or you can give a ready-made AI, like qwen, a well-written prompt."
> "Writing that prompt well takes trial and error for every new app. Our framework automates it: from a description of the app, an AI agent designs the pipeline, tests it and delivers the best one."

If asked:

Question	Answer
What is an intent?	What the user wants to do. 'Turn off the lights' has the intent iot_hue_lightoff.
What is NLP?	Natural language processing: getting computers to work with human language.

Step 2. How the system works (3 min)

Show: the diagram below (also in How Our System Works, section 9).



The framework's steps. Blue = where the AI decides.

Say:

> "You give the framework a description of the app, its list of intents and about 10 example sentences per intent."
> "First the Analyzer: the AI writes a short description of every intent."
> "Then the Architect: there are 12 ways to write the prompt, which we call designs. The AI picks 3 it thinks will work, our program tests each on 100 practice sentences, the AI sees the scores and picks a 4th."
> "The best of the 4 is kept and saved as a package any app can load."
> "Important: qwen itself never changes. We are not training it. We only choose the best way to ask it."

If asked:

Question	Answer
Is this training?	No. Training changes the model, like XLM-R. Here the model stays the same; we only choose the best prompt.
Who makes the pipeline, qwen?	No. Our Python program builds and tests it. qwen gives advice on which designs to try, and later answers each request.
What are the 12 designs?	3 ways to ask × 2 ways to choose examples × 2 ways to refuse off-topic requests.

Step 3. Live demo of the finished pipeline (2 min)

Show: PowerShell.

Type:

```
python scripts/demo.py --project P1
```

Say:

> "This is the pipeline our agent chose for the home-assistant project, loaded fresh as a saved package."
> "It handles normal requests, refuses off-topic ones, and rejects invalid input before even calling the AI."

Point at: '-> ok intent=alarm_set time=six thirty' (understood, with the detail extracted); '-> out_of_scope' (correctly refused); '-> invalid_request' (empty input refused).

If asked:

Question	Answer
Can I try my own sentence?	Yes: run python scripts/demo.py --project P1 --interactive and let the teacher type. Press Enter on an empty line to stop.
Does it need the internet?	No. The AI model runs locally on this laptop.

Step 4. One test, step by step (3 min)

Show: PowerShell.

Type:

```
python scripts/walkthrough.py
```

Say:

> "This shows exactly how every test works, for three requests."
> "Step 1 is the correct answer, from the dataset, labelled by people. Step 3 is the prompt we send to the AI. Step 4 is its raw reply. Step 5 is how we read and check it. Step 6 is whether it was right."
> "A real experiment does exactly this for all 100 practice requests, saves every answer, and computes the F1 score."

If asked:

Question	Answer
Where does the correct answer come from?	data\processed\P1\dev.jsonl: the MASSIVE dataset, labelled by people.
How is F1 computed?	Run python scripts/explain_f1.py: it puts the correct answers and the system's answers side by side and shows every count and the formula. Ours gives 0.974, the same as the recorded result and scikit-learn.

Step 5. The agent choosing a design (2 min)

Show: PowerShell.

Type:

```
python scripts/run_a2.py
```

Say:

> "Here the agent designs the pipeline itself. First the AI ranks each design choice and gives its reasons."
> "Our program builds 3 designs from that, tests them, shows the AI the scores, and the AI changes one thing."
> "Notice the AI's first preferences were partly wrong, but testing caught that: the 4th design scored 0.974, the best of all 12 designs, while testing only 4."

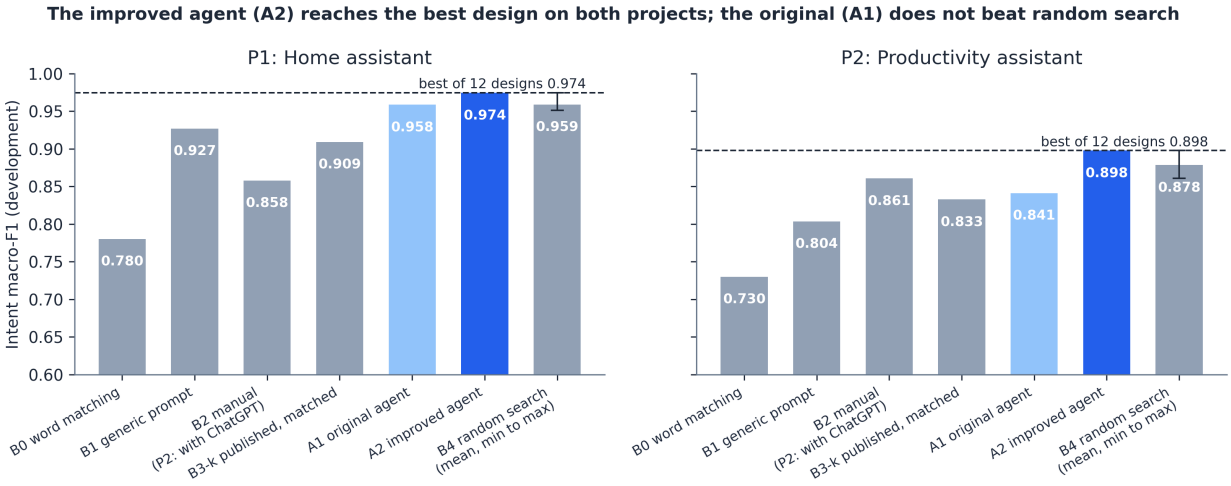
Point at: the 'ranked' lines (the AI's reasoning), the 'tried' lines with their scores, and 'selected schema-sim-llm: intent 0.974'.

If asked:

Question	Answer
What makes it agentic?	The AI makes design decisions, sees the results of its own choices, and revises them, with no human configuring anything.

Step 6. The results and the comparisons (4 min)

Show: reports\figures\fig1_main_comparison.png, then fig2_intent_vs_entity.png.



Intent scores for every system on both projects.

Say:

"To prove it is better, we compared it with other ways of building the pipeline, all on the same requests, with the same AI model and the same scorer."
"B0 is simple word matching. B1 is one standard prompt for every app. B2 is a person doing it by hand. B3 is XLM-R trained on the same examples, the published baseline. B4 picks designs at random."
"Our improved agent, A2, scored 0.974 and 0.898: the best of all 12 designs on both projects. It beats the generic prompt, the person working alone, the published baseline, and random search."
"We also report where it was weaker: our first version, A1, did not beat random search, and that is why we built A2."

If asked:

Question	Answer
Why compare with random search?	To check the agent's choices are smart, not lucky. A1 was no better than random; A2 is.
What is XLM-R?	A language model that has to be trained on examples. With only 10 examples per intent it scored 0.909; ours scored 0.974 without any training.
Are these final results?	These are development scores. The final test results (300 locked requests per project, plus the held-out P3) are in the last section: A2 met the success rule on P1 and P2 and found the best design on all three projects.

Step 7. Proof the numbers are real (2 min)

Show: PowerShell.

Type:

```
python scripts/verify_with_sklearn.py
```

Say:

> "Every answer the systems gave is saved, so any result can be recomputed."
> "This script recomputes our scores with scikit-learn, the standard library, without using our own scoring code. Every run agrees, to 9 decimal places."

Point at: every line says 'agrees'. For everything at once: python scripts/show_evidence.py (ends with 'Overall: all checks passed').

If asked:

Question	Answer
How do I know you did not edit the numbers?	reproduce_metrics.py re-scores every run from its saved answers, and a test proves it catches a hand-edited number.
Did you tune on the test data?	No. The code locks the test data until the freeze, and a test checks it.

Step 8. Built it yourself (optional, 2 min)

Show: diy\c04_b1.py open in VS Code, then PowerShell.

Type:

```
python diy/c04_b1.py
```

Say:

> "To make sure we understand it, we also wrote small versions of each test from scratch."
> "This 40-line script builds a prompt, asks the AI about every request and scores it. It gets 0.927, the same as the project's B1 baseline."

If asked:

Question	Answer
Can you change something?	Yes: in diy\c03_b0.py change 0.35 to 0.5 and run it again; the score changes because more requests are refused.

Step 9. Honest limits and next step (1 min)

Show: How Our System Works, sections 14 and 15.

Say:

> "The final pipeline is always one of the 12 designs, customised for the project. That keeps it safe and lets us prove whether the agent is smart."
> "Limits: one small model, development data so far, and the human baseline is one beginner per project."
> "Next, we plan A3: the agent will start from the best design and rewrite its own prompt from its mistakes, keeping only changes that improve the score."

If asked:

Question	Answer
Why not let it design anything?	Then generated code would run, we could not compare it fairly, and our small 3B model already makes mistakes. Prompt refinement (A3) keeps it safe and measurable.

Step 10. Close (1 min)

Show: nothing; look at the teacher.

Say:

> "To sum up: our framework automates designing an NLP pipeline for any new app. It is tested against a generic prompt, a person, random search and the published baseline, and the results can be recomputed by anyone."
> "We used Claude Code, an AI coding assistant, to write the code and run the experiments; we planned it, checked the results, and can explain and re-run every step. That is recorded in docs\contributions.md."
> "Thank you. We are happy to answer questions or run anything again."

Quick answers for questions

If the teacher asks...	Say	Show
What did you build?	A framework that automatically designs, tests and delivers an NLP pipeline for any new app.	How Our System Works, section 1
How is it different from XLM-R?	XLM-R learns by training; our framework designs the best prompt for a model that never changes.	section 4 diagram
What does qwen do?	Gives advice while designing, and answers each request once the pipeline runs.	section 6
What is in the prompt?	The intent list, short descriptions, real example sentences and the request.	section 7
How was each test done?	Same 100 requests, same model, same scorer; every answer saved.	python scripts/walkthrough.py
Show me F1 by hand.	Correct vs system answers, counted per intent, averaged.	python scripts/explain_f1.py
Is the difference real?	On test, A2's gain over B1 is significant on P2 (+4.9 points) and its entity gains on all three projects; on P1 (+2.2) the interval still includes zero.	python scripts/explain_stats.py bootstrap
How fast is it?	Under one second per request.	python scripts/explain_stats.py timing
What mistakes does it make?	Mostly extracting extra words as details.	python scripts/explain_stats.py errors
Run all your tests.	371 automated tests.	python -m pytest tests
Is it innovative?	It automates the whole design and is rigorously compared; A2 shows structured search matters. Next: A3 refinement.	section 14
Did you use AI?	Yes, Claude Code wrote the code; we planned, checked and can explain everything. It is recorded.	docs\contributions.md

Backup if the live demo fails

- Open the documents instead: **How_Every_Test_Was_Done_v2.pdf** shows the real output of every command.
- Open **reports\EVIDENCE_SUMMARY.md**: the saved output of all checks.
- Charts are in **reports\figures**; they do not need the model.

More questions and answers

Grouped by topic. Each answer is short enough to say aloud; the extra detail is there if the teacher asks more.

Using AI tools instead

Question	Answer
Why not just use an AI tool like ChatGPT to make the pipeline?	We tested that. On P2 one of us used ChatGPT to design the pipeline (B2): it scored 0.861, slightly above our first agent (0.841) but below our improved agent (0.898), which needed no human. Show: python scripts/explain_f1.py --run 20261024_1630_P2_B2_0. It is one run on one project, so it suggests rather than proves.
So is an AI tool better or worse?	An AI tool gives an opinion; our framework tests every option on real requests and keeps the one that scores best. Even inside our system, qwen's first advice was partly wrong and testing caught it. Our framework is also automatic, repeatable, recorded, and runs offline.
Are there automatic tools that already do this?	Yes, for example DSPy, which tunes prompts automatically, and Prompt2Model. They are in our related work. Our plan listed a DSPy comparison as optional (B5) and it has not been done yet; we plan to add it.
Why not call the ChatGPT API at runtime instead of a small local model?	Cost per request, privacy (user messages would leave the device), and dependence on an internet service. The plan also requires one model for every compared system, so the comparison stays fair; a local model makes that possible at zero cost.

Who would use it, and real-world use cases

Question	Answer
Who is going to use this?	Developers and small teams building any app that has to understand short user requests, especially when they have only a few examples and no machine-learning expert. They describe the app, give about 10 examples per request type, and get a tested pipeline back.
Give a real example.	A hospital app: patients type 'I need to see a cardiologist on Monday' or 'cancel my appointment'. The developer lists the request types (book, cancel, check bed availability...) with a few examples each; the framework designs and tests the pipeline, and the app gets intent = book_appointment, details = cardiologist, Monday.
Which industries could use it?	Healthcare (appointments, symptom triage routing); banking (check balance, block card, report fraud); e-commerce (track order, return item); telecom (recharge, complaint about network); smart homes and devices (like our P1); office tools (calendar, email, lists, like our P2); travel and food ordering (like our held-out P3); HR and IT helpdesks (reset password, apply for leave); government services; education (course queries, fee payment).
Where would it sit in a real product?	Between the user and the app's actions: a chat box or voice assistant sends the text to the exported pipeline, which returns the intent and details; the app then books, cancels or answers. Our demo shows that step.
What makes it useful for a company?	Less setup time (no hand-tuning for every new app), a measured quality score before going live, and it can run on the company's own computer, so customer data stays private.

How well it works

Question	Answer
Is the accuracy good enough for a real product?	On development data the improved agent gets 0.974 on the home assistant and 0.898 on the productivity assistant. For a real product you would test on that product's own messages first, and add a fallback such as asking the user to rephrase when it refuses a request.
What happens when it gives a wrong answer?	Every reply is checked: a broken reply is caught and retried once; an unclear request can be refused rather than acted on wrongly. The project description itself says acting on the wrong device is worse than admitting the request was not understood.
How much data does it need?	About 10 labelled example sentences per request type, plus some labelled requests to test on. That is the realistic case for a new app, and it is where training a model like XLM-R struggled (0.909 against our 0.974).
Does it handle messy typing?	Yes, we tested typos, capitals and filler words: the AI systems lost at most about 7 points, while simple word matching lost up to 27. Show: reports\figures\fig6_robustness.png.
How fast is it?	Under one second per request on this laptop, measured live. Show: python scripts/explain_stats.py timing.
Does it work in other languages?	We tested English only. MASSIVE has 51 languages and qwen is multilingual, so it should extend, but that is untested and would need its own evaluation.

Design choices

Question	Answer
Why the MASSIVE dataset?	It is public (CC BY 4.0), labelled by people, has many domains to build separate projects from, and comes with a published baseline (XLM-R) to compare against.
Why qwen2.5:3b?	It fits the laptop's 4 GB graphics card, answered in about 0.4 s, always replied in valid JSON and gave identical answers on repeat. A larger 8B model was too big for the card.
Why only 12 designs?	Safety (no AI-written code runs), fair proof (we can compare with random search, a person and all 12), and a small model that already makes mistakes. Next step: A3 lets the agent rewrite its own prompt.
Why test only 4 of the 12?	Testing costs time; a real system cannot try everything. Choosing well within a small budget is the agent's job, and A2 found the best design while testing 4.
What if the app gets a new request type later?	Add it to the list with about 10 examples and run the framework again: it re-analyses, re-tests and exports a new pipeline. No manual re-tuning.
Can it run without a graphics card?	Yes, more slowly: Ollama can run the model on the processor. We measured speed only on this laptop.

Honesty and process

Question	Answer
What are the weaknesses?	One small model; on the held-out P3 the agent did not beat the generic prompt on intents (−1.3 points, not significant); the human baseline is one beginner per project; the first agent did not beat random search; entity extraction is the weakest area; English only; the final pipeline is always one of 12 designs.
Why is the first agent (A1) worse than random?	Its AI kept choosing one kind of example and never tried the best design. A2 fixed this by structuring the search, with the same model.
Did you change the system after seeing results?	Yes, once: A2, recorded as decision 42 with a keep-or-drop rule fixed in advance. Both A1 and A2 were frozen and reported on the locked test data; P3 was never seen during development.
How do we know you did not cheat?	Test data is locked by code; every answer is saved; scores are recomputed by our scripts and by scikit-learn; and the git history shows the work step by step since 11 October.
What did you do yourselves, if AI wrote the code?	We planned the project, decided what to test and compare, did the human baseline (B2), checked every result, and can explain and re-run each step. The AI assistance is recorded in docs\contributions.md.
What would you do with more time?	A3 (the agent refines its own prompt from its mistakes), a larger development set so choices transfer better (the P3 lesson), the DSPy comparison, more languages, and a larger model.

Final results on the locked test data

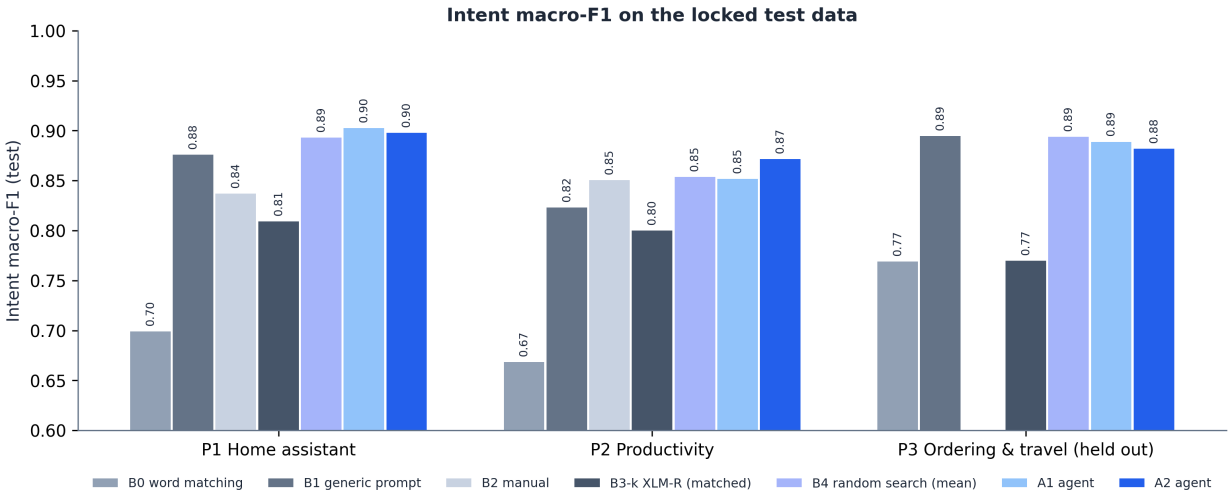
> **Read this section for the final numbers.** Everything above was written during development and uses development scores (100 practice requests). On 25 October 2026 the protocol, every system's choice and all code were frozen in one commit (`0ee6b22b`); only then were the test requests and the held-out project P3 unlocked. These are the results reported in the paper.

What was tested

- P1 home assistant and P2 productivity: 300 test requests each (250 in scope + 50 off-topic).
- P3 ordering and travel, **held out**: never used during development; 200 test requests (167 + 33).
- Every system's design was chosen on development data before the freeze and replayed, not re-chosen, on test.

The results (intent F1 / entity F1 / off-topic F1)

System	P1	P2	P3 (held out)
B0 word matching	0.699 / – / 0.426	0.669 / – / 0.403	0.769 / – / 0.404
B1 generic prompt	0.876 / 0.626 / 0.875	0.823 / 0.477 / 0.662	0.895 / 0.529 / 0.771
B2 person	0.837 / 0.648 / 0.471	0.851 / 0.491 / 0.748	not run
B3-k XLM-R, same examples	0.809 / 0.650 / 0.568	0.800 / 0.461 / 0.536	0.770 / 0.515 / 0.505
B3-full XLM-R, all training data	0.883 / 0.802 / 0.604	0.904 / 0.775 / 0.560	0.913 / 0.721 / 0.581
B4 random search (mean intent, range)	0.893 (0.881–0.903)	0.854 (0.848–0.872)	0.894 (0.882–0.909)
A1 first agent	0.903 / 0.639 / 0.885	0.852 / 0.504 / 0.758	0.889 / 0.506 / 0.845
A2 improved agent	0.898 / 0.699 / 0.865	0.872 / 0.532 / 0.767	0.882 / 0.675 / 0.795



Intent F1 on the locked test data for every system and project.

What the results mean

- **A2 found the best design every time.** On all three projects it picked the design that is best of all 12 on development data, while testing only 4.
- **A2 beat the generic prompt (B1) on P1 and P2 and met the success rule.** Intent difference in points with 95% interval: P1 +2.2 [-0.6, +5.1], P2 +4.9 [+1.1, +8.9]. Entity: P1 +7.3 [+0.8, +14.5], P2 +5.5 [+0.3, +11.1]. The P2 intent gain and both entity gains are significant (interval above zero).
- **On the held-out P3 it did not beat B1 on intents:** -1.3 [-3.8, +1.4] points, an interval that includes zero, so it could be luck. On entities it was much better: +14.6 [+9.0, +21.3]. P3's practice set had only about 13 requests per intent, and the designs were very close on test.
- **Against the person (B2):** A2 was +6.1 [+2.0, +10.2] points higher on P1 (significant) and +2.1 [-0.7, +5.3] on P2 (not significant). On P1 the person chose the similarity threshold for off-topic requests, and off-topic F1 fell to 0.471.
- **Against XLM-R with the same 10 examples per intent (B3-k):** A2 was 7 to 11 points higher on intents, higher on entities and far better at refusing off-topic requests on every project. With 10 to 32 times more labelled data (B3-full), XLM-R is better at entities but still refuses badly.
- **Against random search (B4):** A2 was above the random average on P1, at the top of the random range on P2, and at the bottom of the range on P3.
- **A1** passed the success rule on P1 and P2 too, but none of its differences from B1 is significant, and it failed on P3.

Speed and reliability

- 95th-percentile time per request (limit = 2 × B1 in the same session): P1 A2 0.89 s (limit 1.22 s), P2 A2 1.08 s (limit 1.67 s). Median about 0.65 s.
- Valid replies after repair: at least 0.997 for every agent run (needed 0.98).
- Designing one project costs A2 about 397 model calls, about 3 minutes, against 38 minutes for the person on P1.

Proof the numbers are real

- `python scripts/reproduce_metrics.py`: 99 of 99 runs recomputed from their saved answers match exactly.
- `python scripts/verify_with_sklearn.py`: scikit-learn recomputes the same scores to 9 decimal places.
- `python scripts/reproduce_live.py --freeze-commit 0ee6b22b86894aebbe253fae3b00eb97c542c575`: B0 retrained from scratch is identical, and A2 asked live with the cache off gave 50 of 50 answers identical to the recorded test answers.
- `python scripts/show_evidence.py` runs every check and the live demo in one go (`reports/EVIDENCE_SUMMARY.md`).

Full details: the paper (`paper/main.tex` and its PDF), `paper/Research_Paper_Explained.pdf`, `docs/REPRODUCTION.md` and `reports/final_test.json`.